Autonomous and Mobile Robotics

Midterm Class Test Exercises: Hopping Robot, Chained Form Planning, Front-Wheel-Drive Bicycle

Lecture notes

Contents

1 Toolbox: the Recurring Ideas **1**
- 1.1 Pfaffian constraints and kinematic models . . . 1
- 1.2 Controllability by Lie brackets . . . 2
- 1.3 Differential flatness and reconstruction . . . 2
- 1.4 Dynamic feedback linearization . . . 2

2 The Single-Legged Hopping Robot **2**
- 2.1 Setting . . . 3
- 2.2 (a) Geometry of the configuration space . . . 4
- 2.3 (b) A kinematic model with physically meaningful inputs . . . 4
- 2.4 (c) Local and global mobility . . . 5
- 2.5 (d) Feedback control in flight: $\theta \to 0$, $\ell \to h$ . . . 6
- 2.6 (e) Differential flatness for $d = 0$ and path planning . . . 7

3 Planning for a $(2,3)$ Chained Form **9**
- 3.1 Setting . . . 9
- 3.2 (a) The algorithm . . . 10
- 3.3 (b) Free parameters and how to choose them . . . 11
- 3.4 (c) The case $\boldsymbol{z}_i = (0,0,0)$, $\boldsymbol{z}_f = (1,1,1)$ . . . 11

4 Front-Wheel-Drive Bicycle: Modelling, Flatness and Control **12**
- 4.1 Setting . . . 13
- 4.2 (a) Kinematic model and controllability . . . 13
- 4.3 (b) Are x_f, y_f flat outputs? . . . 14
- 4.4 (c) Tracking controller for (x_f, y_f): dynamic feedback linearization . . . 15

5 Summary **17**
- 5.1 Comparison of the three problems . . . 17
- 5.2 Key points . . . 17

These notes work out in full three exercises on the kinematic modelling, controllability, flatness, planning and control of nonholonomic robots:

- the single-legged **hopping robot** in flight phase (Midterm 2022/23, Problem 1);
- **planning for a** $(2,3)$ **chained form** with a two-phase algorithm (Midterm 2021/22, Problem 2);
- the **front-wheel-drive bicycle**: kinematic model, controllability, (non-)flatness of the front wheel position and dynamic feedback linearization (Midterm 2024/25, Problem 1).

Each problem is solved part by part. Every non-obvious step is followed by a blue *Intuition* box.

1 Toolbox: the Recurring Ideas

All three exercises are driftless nonholonomic systems, so the same few ideas keep returning. We collect them here.

1.1 Pfaffian constraints and kinematic models

A *kinematic constraint* is linear in the generalized velocities,

$$A^T(\boldsymbol{q})\,\dot{\boldsymbol{q}} = 0, \qquad A(\boldsymbol{q}) \in \mathbb{R}^{n\times k}, \tag{1}$$

where $n = \dim \boldsymbol{q}$ and k is the number of (independent) constraints. All admissible velocities lie in the $(n-k)$-dimensional null space of $A^T(\boldsymbol{q})$. Choosing a basis $\boldsymbol{g}_1(\boldsymbol{q}), \ldots, \boldsymbol{g}_{n-k}(\boldsymbol{q})$ of this null space gives the *kinematic model*

$$\dot{\boldsymbol{q}} = \sum_{j=1}^{n-k} \boldsymbol{g}_j(\boldsymbol{q})\, u_j = G(\boldsymbol{q})\, u. \tag{2}$$

Intuition

The constraint says "you may not move in these k directions" (a wheel cannot skid sideways; angular momentum cannot be created from nothing). What is left is a space of allowed velocities, and the kinematic model simply parametrizes it with $n-k$ free numbers u_j. The basis is *not unique*: any invertible u-reparametrization gives an equivalent model. The good choice is the one in which the u_j are quantities you can actually command, for instance joint velocities.

1.2 Controllability by Lie brackets

For driftless systems, controllability is decided by the *accessibility rank condition*: if the involutive closure of $\operatorname{span}\{\boldsymbol{g}_1, \ldots, \boldsymbol{g}_{n-k}\}$ (obtained by adding Lie brackets $[\boldsymbol{g}_i, \boldsymbol{g}_j] = \frac{\partial \boldsymbol{g}_j}{\partial \boldsymbol{q}}\boldsymbol{g}_i - \frac{\partial \boldsymbol{g}_i}{\partial \boldsymbol{q}}\boldsymbol{g}_j$, then brackets of brackets, ...) has dimension n at every $\boldsymbol{q}$, the system is controllable (Chow's theorem). If a constraint is *nonholonomic* it is not integrable, so no loss of reachable configurations occurs: local velocity restrictions do not restrict global mobility.

Intuition

Think of parallel parking. You cannot slide the car sideways (a velocity restriction), yet you can reach any position by wiggling forward and backward while steering. The Lie bracket $[\boldsymbol{g}_i, \boldsymbol{g}_j]$ is the direction obtained by the sequence "move along $\boldsymbol{g}_i$, then $\boldsymbol{g}_j$, then back along $\boldsymbol{g}_i$, then back along $\boldsymbol{g}_j$"; to first order the net displacement is along the bracket. Brackets are therefore the "wiggle directions" that are not available directly.

1.3 Differential flatness and reconstruction

A system $\dot{\boldsymbol{q}} = G(\boldsymbol{q})u$ is *differentially flat* with *flat outputs* y if y has as many components as there are inputs, and $\boldsymbol{q}$ and u can be reconstructed *algebraically* (i.e. without integrating any differential equation) from y and a finite number of its derivatives. For driftless systems, the *geometric model* is obtained by replacing $\dot{(\cdot)}$ with $(\cdot)' = \mathrm{d}(\cdot)/\mathrm{d}s$ with respect to a path parameter s, and inputs u by geometric inputs $\tilde{u}$ (so that $u = \tilde{u}\,\dot{s}$). If y is flat, a feasible path is planned by interpolating $y(s)$ with sufficiently smooth functions (e.g. polynomials) that match the boundary conditions, then reconstructing $\boldsymbol{q}(s)$ and $\tilde{u}(s)$.

Intuition

"Flat" means the whole robot behaviour is encoded in a few free functions. Once you pick the curves $y(s)$ freely, nothing else is left to choose: the remaining states and inputs follow by differentiating and solving algebraic equations. This turns a planning problem with a non-holonomic constraint into a simple interpolation problem. The condition to check is whether the *hidden* state variables can be recovered from y without knowing an initial condition. If recovering a variable requires solving an ODE, it carries an arbitrary initial condition and y is *not* flat.

1.4 Dynamic feedback linearization

When the outputs to be controlled are such that the input does not appear in their first derivatives (or a static decoupling matrix is singular), one may *extend the dynamics* by integrating some inputs. If after the extension the *decoupling matrix* $\boldsymbol{T}$ in

$$y^{(r)} = \boldsymbol{b}(\boldsymbol{q}, \xi) + \boldsymbol{T}(\boldsymbol{q}, \xi)\, \tilde{u} \tag{3}$$

is invertible, the choice $\tilde{u} = \boldsymbol{T}^{-1}(w - \boldsymbol{b})$ turns the system into $y^{(r)} = w$, i.e. decoupled chains of integrators, for which linear PD+feedforward tracking is available.

2 The Single-Legged Hopping Robot

(Midterm Class Test 2022/23, Problem 1)

Problem statement

A simplified model of a single-legged hopping robot consists of a body with inertia moment J and a leg of mass m. The leg is connected to the body through a revolute joint and a prismatic joint, which are separated by a distance d. Both joints are actuated; in particular, the extension of the length can be changed via the prismatic joint actuator.
When the robot is in flight phase, angular momentum is conserved. This leads to the following constraint

$$J\dot{\theta} + m(d + \ell)^2 \dot{\phi} = 0,$$

where we have assumed that the leg mass is concentrated at its extremity (foot) and the initial momentum is zero. Let the configuration of the robot be defined as $\boldsymbol{q} = (\phi, \ell, \theta)$ (the translational motion is not of interest).

(a) Discuss the geometry of the robot configuration space.
(b) Derive a kinematic model of the robot for which there is a clear physical interpretation of the associated inputs.
(c) Discuss local and global mobility of the robot in its configuration space.
(d) Using the kinematic model, find a feedback control law for the flight phase that will drive the body orientation θ to zero and the leg length to a constant value h. Provide a block scheme.
(e) Consider for simplicity the case $d = 0$. Show that the system is differentially flat and derive the associated reconstruction formulas. Based on this, devise an algorithm for planning a feasible path between two arbitrary configurations $\boldsymbol{q}_s$ and $\boldsymbol{q}_g$.

2.1 Setting

A simplified single-legged hopping robot consists of a body with inertia moment J and a leg of mass m concentrated at its extremity (the foot). The leg is attached to the body through a revolute joint and a prismatic joint, separated by a distance d; both are actuated, so the leg

length ℓ can be changed. Let ϕ be the leg angle (relative to the horizontal), ℓ the leg extension and θ the body orientation. Only the orientation-related motion is of interest, so

$$\boldsymbol{q} = (\phi, \ell, \theta)^T.$$

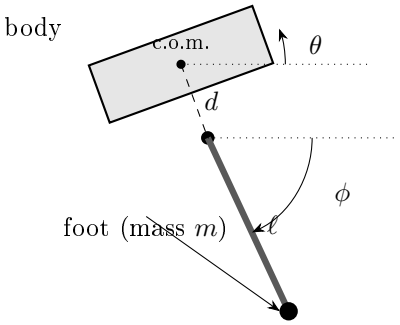


Figure 1: Schematic of the hopping robot: body with inertia J and orientation θ, revolute joint (angle ϕ) and prismatic joint (extension ℓ) separated by d. Original redrawing.

During the flight phase the total angular momentum is conserved. With zero initial momentum and the leg mass concentrated at the foot,

$$J\dot{\theta} + m(d+\ell)^2\dot{\phi} = 0. \tag{4}$$

Intuition

This is the same principle that lets a falling cat turn in mid-air, or an astronaut reorient without touching anything: with no external torque, total angular momentum stays zero, so rotating the leg one way forces the body to rotate the other way. How *much* the body turns depends on the lever arm $(d+\ell)$ of the leg: since $\dot{\theta} = -\frac{m(d+\ell)^2}{J}\dot{\phi}$, a *longer* leg produces *more* body rotation for the same swing. This lever-arm effect is what makes the robot controllable in part (c).

2.2 (a) Geometry of the configuration space

Both ϕ and θ are angles, hence elements of SO(2) (circles), whereas the extension ℓ is a real number (physically $\ell \geq 0$). Therefore

$$\mathcal{C} = \mathrm{SO}(2) \times \mathbb{R} \times \mathrm{SO}(2), \qquad n = \dim \mathcal{C} = 3.$$

There is a single constraint, (4), so $k = 1$.

Intuition

The configuration space is not $\mathbb{R}^3$ but a "cylinder times a circle": angles wrap around (ϕ and $\phi + 2\pi$ are the same configuration). This matters later, when planning: reaching ϕ_g may be done by choosing any representative $\phi_g + 2\pi k$, which changes the path.

2.3 (b) A kinematic model with physically meaningful inputs

Writing (4) in Pfaffian form,

$$\underbrace{\begin{pmatrix} m(d+\ell)^2 & 0 & J \end{pmatrix}}_{A^T(\boldsymbol{q})} \begin{pmatrix} \dot{\phi} \\ \dot{\ell} \\ \dot{\theta} \end{pmatrix} = 0. \tag{5}$$

The null space of $A^T(\boldsymbol{q})$ has dimension $n-k=2$. Two vector fields spanning it are

$$\boldsymbol{g}_1(\boldsymbol{q}) = \begin{pmatrix} 1 \\ 0 \\ -\frac{m(d+\ell)^2}{J} \end{pmatrix}, \qquad \boldsymbol{g}_2(\boldsymbol{q}) = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, \tag{6}$$

which gives the kinematic model

$$\boxed{\begin{aligned} \dot{\phi} &= u_1 \\ \dot{\ell} &= u_2 \\ \dot{\theta} &= -\frac{m(d+\ell)^2}{J}\, u_1. \end{aligned}} \tag{7}$$

How the basis is found. We want $\dot{\phi}$ and $\dot{\ell}$ to be the inputs, since they are the velocities of the two actuated joints. Hence the first two components of the basis vectors should be $(1,0)$ and $(0,1)$. The third component α of $\boldsymbol{g}_1 = (1,0,\alpha)^T$ is fixed by requiring $A^T\boldsymbol{g}_1 = 0$:

$$m(d+\ell)^2 + J\alpha = 0 \implies \alpha = -\frac{m(d+\ell)^2}{J},$$

while $\boldsymbol{g}_2 = (0,1,0)^T$ is automatically in the null space since the constraint does not involve $\dot{\ell}$.

Intuition

The model says: command the two joints freely (u_1 swings the leg, u_2 extends it), and the body orientation is *slaved* to the leg swing through the conservation of angular momentum. $\dot{\ell} = u_2$ does not affect θ at all, as extending the leg exerts no torque about the center of mass in this model, but *when* you swing depends on ℓ. Any other basis is equally valid mathematically (e.g. taking $\dot{\theta}$ as an input), but then the inputs would no longer be joint velocities.

2.4 (c) Local and global mobility

Local mobility. At each $\boldsymbol{q}$, the velocity $\dot{\boldsymbol{q}}$ must belong to $\mathcal{N}(A^T(\boldsymbol{q}))$, a 2-dimensional subspace of the 3-dimensional tangent space. So *locally* the robot cannot move in an arbitrary direction: the body cannot rotate independently of the leg.

Global mobility. We check the accessibility rank condition. The only non-trivial bracket is

$$\boldsymbol{g}_3(\boldsymbol{q}) = [\boldsymbol{g}_1, \boldsymbol{g}_2](\boldsymbol{q}) = \frac{\partial \boldsymbol{g}_2}{\partial \boldsymbol{q}}\boldsymbol{g}_1 - \frac{\partial \boldsymbol{g}_1}{\partial \boldsymbol{q}}\boldsymbol{g}_2 = -\frac{\partial \boldsymbol{g}_1}{\partial \ell} = \begin{pmatrix} 0 \\ 0 \\ \frac{2m(d+\ell)}{J} \end{pmatrix}. \tag{8}$$

Its third component never vanishes provided $d+\ell > 0$ (this holds for $\ell \geq 0$ as soon as $d > 0$; if $d = 0$ the point $\ell = 0$ is singular, where the leg has no lever arm). Hence

$$\operatorname{rank}\begin{pmatrix} \boldsymbol{g}_1 & \boldsymbol{g}_2 & \boldsymbol{g}_3 \end{pmatrix} = 3,$$

and the kinematic model is *controllable*. In other words, the angular-momentum conservation constraint is *nonholonomic*: global mobility is not restricted, and every configuration of $\mathcal{C}$ can be reached.

Intuition

Here is the concrete "wiggle" behind the bracket. Take $m/J = 1$, $d = 0$. Extend the leg to $\ell_2 = 2$, swing it by $\Delta\phi = 0.5\,\text{rad}$, retract to $\ell_1 = 1$, and swing back by $-0.5\,\text{rad}$. The body rotation produced while swinging out is $-\ell_2^2\Delta\phi = -2\,\text{rad}$; the one while swinging back is $+\ell_1^2\Delta\phi = +0.5\,\text{rad}$. The leg is back at the same (ϕ, ℓ), but the body has rotated by

$$\Delta\theta = -\frac{m}{J}\Delta\phi\left[(d+\ell_2)^2 - (d+\ell_1)^2\right] = -0.5\,(4-1) = -1.5\ \text{rad}.$$

So a closed cycle in the leg variables (ϕ, ℓ) produces a net rotation of the body: this is why one can reorient arbitrarily in flight (exactly like the cat), and it is the geometric meaning of $\boldsymbol{g}_3$ pointing along θ.

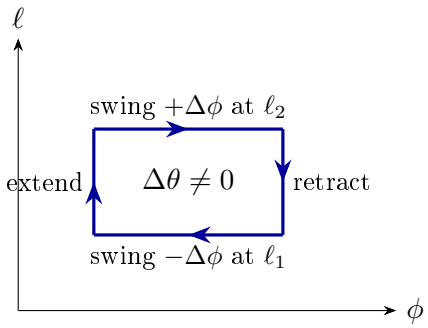


Figure 2: A rectangular loop in the leg variables (ϕ, ℓ) leaves the leg where it started but produces a net body rotation $\Delta\theta$ (nonholonomy).

2.5 (d) Feedback control in flight: $\theta \to 0$, $\ell \to h$

In the model (7) the leg length is a pure integrator, $\dot{\ell} = u_2$. Choosing

$$u_2 = k_\ell\,(h - d - \ell), \qquad k_\ell > 0, \tag{9}$$

gives $\dot{\ell} = -k_\ell(\ell - (h-d))$, i.e. exponential convergence of ℓ to $h-d$, so that the total leg length $d+\ell$ converges to the desired h.

For θ, introduce the input transformation (always invertible, since $d + \ell > 0$)

$$u_1 = -\frac{J}{m(d+\ell)^2}\,v_1,$$

under which $\dot{\theta} = v_1$. Taking $v_1 = -k_\theta\theta$, $k_\theta > 0$, yields exponential convergence of θ to zero. Going back to the original input:

$$u_1 = k_\theta\,\frac{J}{m(d+\ell)^2}\,\theta. \tag{10}$$

The control law (9)–(10) only needs measurements of ℓ and θ (not ϕ).

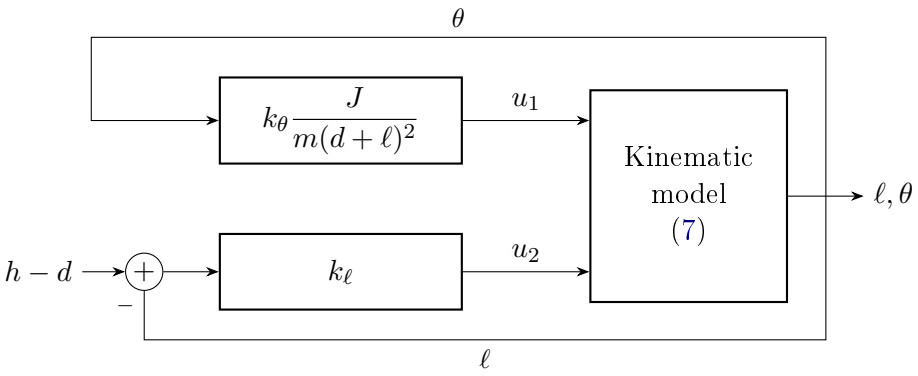


Figure 3: Block scheme of the flight-phase controller: two decoupled loops. The θ-loop applies the state-dependent gain $k_\theta J/(m(d+\ell)^2)$ to θ (regulation to zero); the ℓ-loop is a proportional regulator on $h - d - \ell$.

Intuition

The trick is to make each variable look like a trivial first-order system $\dot{x} = -kx$. For ℓ this is immediate. For θ, the factor $\frac{m(d+\ell)^2}{J}$ is a (known, positive) "gear ratio" between the swing velocity u_1 and the body rotation rate, so we simply divide by it. The price is that ϕ is left uncontrolled: it drifts. How far? If ℓ were constant, momentum conservation gives $J\Delta\theta = -m(d+\ell)^2\Delta\phi$. Straightening the body from $\theta_0 = 0.3\,\text{rad}$ with $d + \ell = h = 0.5\,\text{m}$, $m/J = 2\,\text{m}^{-2}$ requires the leg to rotate by

$$\Delta\phi = \frac{J}{mh^2}\theta_0 = \frac{0.3}{2 \cdot 0.25} = 0.6 \text{ rad}.$$

2.6 (e) Differential flatness for $d = 0$ and path planning

For $d = 0$ the model (7) becomes

$$\begin{aligned} \dot{\phi} &= u_1 \\ \dot{\ell} &= u_2 \\ \dot{\theta} &= -\frac{m\ell^2}{J}u_1. \end{aligned} \tag{11}$$

Proposition 2.1. *System (11) is differentially flat with flat outputs (ϕ, θ).*

Proof. Using $u_1 = \dot{\phi}$ in the last equation gives $\ell^2 = -\frac{J}{m}\frac{\dot{\theta}}{\dot{\phi}}$. Since $\ell \geq 0$,

$$\ell = \sqrt{-\frac{J}{m}\frac{\dot{\theta}}{\dot{\phi}}}. \tag{12}$$

The inputs follow immediately:

$$u_1 = \dot{\phi}, \qquad u_2 = \dot{\ell} = \frac{1}{2\ell}\left(-\frac{J}{m}\right)\frac{\ddot{\theta}\dot{\phi} - \dot{\theta}\ddot{\phi}}{\dot{\phi}^2} = \frac{J}{2m\ell}\frac{\dot{\theta}\ddot{\phi} - \ddot{\theta}\dot{\phi}}{\dot{\phi}^2}, \tag{13}$$

with ℓ given by (12). All reconstruction formulas are algebraic in (ϕ, θ) and their derivatives (up to second order), so (ϕ, θ) is a flat output. □

Remark 2.2. Constraint (4) implies that $\dot{\theta}$ and $\dot{\phi}$ always have *opposite* signs, so the radicand in (12) is nonnegative: the square root is well-posed, and $\ell > 0$ is consistent with the constraint. The formulas require $\dot{\phi} \neq 0$.

Intuition

Why is ℓ *not* a flat output while θ is? Because ℓ can be read off from the *ratio* of how fast the body turns to how fast the leg swings: a long leg turns the body a lot per unit swing. So watching $\phi(t)$ and $\theta(t)$ is enough to infer the leg length at every instant, and then its rate, and the input.

Planning. For path planning we use the geometric version of (11), where $(\cdot)' = \mathrm{d}/\mathrm{d}s$:

$$\phi' = \tilde{u}_1, \qquad \ell' = \tilde{u}_2, \qquad \theta' = -\frac{m\ell^2}{J}\tilde{u}_1. \tag{14}$$

Time has disappeared: only the *shape* of the path matters. The reconstruction formulas hold with s-derivatives:

$$\ell = \sqrt{-\tfrac{J}{m}\tfrac{\theta'}{\phi'}}, \qquad \tilde{u}_1 = \phi', \qquad \tilde{u}_2 = \ell' = \frac{J}{2m\ell}\,\frac{\theta'\phi'' - \theta''\phi'}{\phi'^2}.$$

Algorithm: path planning between $\boldsymbol{q}_s = (\phi_s, \ell_s, \theta_s)$ and $\boldsymbol{q}_g = (\phi_g, \ell_g, \theta_g)$ $(d = 0)$

1: Choose a **linear** polynomial $\phi(s) = as + b$, $s \in [0, 1]$, with $\phi(0) = \phi_s$, $\phi(1) = \phi_g$.
Hence $b = \phi_s$, $a = \phi_g - \phi_s$, and $\tilde{u}_1(s) = \phi'(s) = a$ for all s.
2: Choose a **cubic** polynomial $\theta(s) = cs^3 + ds^2 + es + f$ with
$\theta(0) = \theta_s$, $\theta(1) = \theta_g$,
$\theta'(0) = -\dfrac{m\,\ell_s^2}{J}\,a, \quad \theta'(1) = -\dfrac{m\,\ell_g^2}{J}\,a.$
3: Reconstruct $\ell(s) = \sqrt{-\tfrac{J}{m}\tfrac{\theta'(s)}{a}}$ and $\tilde{u}_2(s) = \ell'(s)$.
4: (Optional) Pick a time law $s(t)$ to obtain velocities $u = \tilde{u}\,\dot{s}$.

The boundary conditions on θ' come from the last equation of (14) with $\tilde{u}_1 = a$: they *encode the initial and final leg lengths*. Solving the four linear conditions on the cubic, with $\Delta\theta = \theta_g - \theta_s$ and θ'_s, θ'_g the two prescribed slopes,

$$f = \theta_s, \quad e = \theta'_s, \quad d = 3\Delta\theta - 2\theta'_s - \theta'_g, \quad c = \theta'_s + \theta'_g - 2\Delta\theta.$$

Intuition

Why a line for ϕ and a cubic for θ? ϕ only has two boundary conditions (start and end value), so a straight line suffices, and conveniently gives a *constant* geometric input a. θ has four conditions (two values and two slopes), so we need a cubic. The two slope conditions are what carry the information about ℓ_s and ℓ_g: they say "the body must be turning, at the start, at the rate corresponding to leg length ℓ_s, and at the end at the rate corresponding to ℓ_g". Once $\theta(s)$ is fixed, the leg length in between is whatever is needed to make the constraint hold.

Remark 2.3 (Feasibility check). Since $\ell^2 = -\frac{J}{m}\frac{\theta'}{a}$ must be positive along the whole path, the cubic must satisfy $a\,\theta'(s) < 0$ for all $s \in [0, 1]$: its derivative must not change sign. The boundary slopes already have the right sign; if the interior of θ' crosses zero, the plan is not feasible and one must modify it (e.g. choose $\phi_g + 2\pi k$, i.e. a different number of leg turns, or use a higher-degree polynomial). If $\phi_s = \phi_g$ then $a = 0$ and the formulas degenerate (a nonconstant $\phi(s)$ must be chosen).

Example 2.4 (Numerical example)**.** Take $m/J = 1$, $\phi : 0 \to 1$ (so $a = 1$), $\ell_s = 1$, $\ell_g = 1.5$, and $\theta_s = 0$, $\theta_g = -2$. Then $\theta'_s = -1$, $\theta'_g = -2.25$, $\Delta\theta = -2$, so

$$c = -1 - 2.25 + 4 = 0.75, \quad d = -6 + 2 + 2.25 = -1.75, \quad e = -1, \quad f = 0,$$

$$\theta(s) = 0.75s^3 - 1.75s^2 - s, \qquad \ell(s) = \sqrt{-\theta'(s)} = \sqrt{1 + 3.5s - 2.25s^2}.$$

Here $\theta'(s) = 2.25s^2 - 3.5s - 1$ has its roots at $s \approx -0.25$ and $s \approx 1.8$, outside $[0, 1]$, so $\theta' < 0$ throughout and the path is feasible; indeed $\ell(0) = 1$ and $\ell(1) = 1.5$.

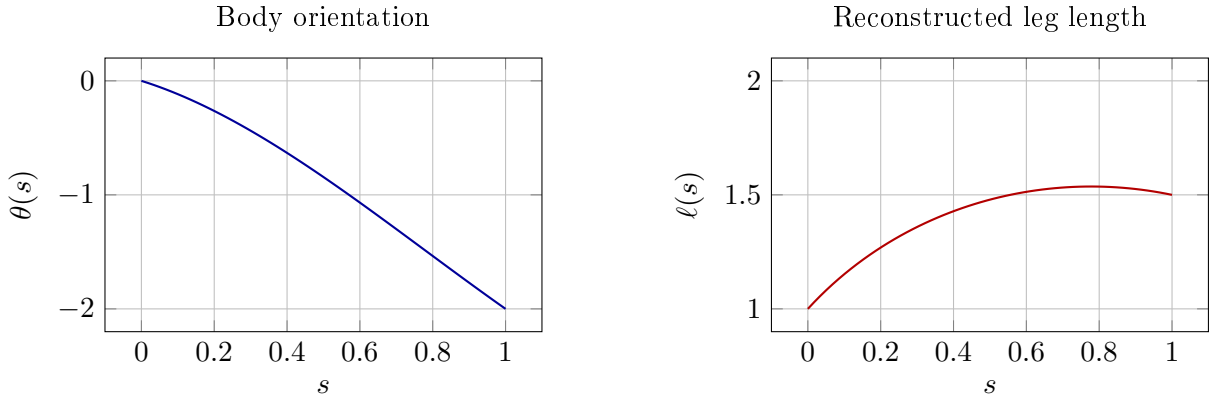


Figure 4: Planned path for the numerical example: the cubic $\theta(s)$ and the leg length $\ell(s)$ reconstructed from it. Note how $\ell(s)$ is never freely chosen: it is imposed by θ.

3 Planning for a $(2, 3)$ Chained Form

(Midterm Class Test 2021/22, Problem 2)

Problem statement

Consider a $(2, 3)$ chained form. It is easy to verify that the application of the control inputs

$$v_1 = a \sin \omega t, \qquad v_2 = b \cos \omega t$$

over the time interval $[0, 2\pi/\omega]$ brings z_1, z_2 back to their initial values and causes a displacement of z_3 equal to $ab\pi/\omega^2$. Exploiting this fact (no need to prove it), design a 2-phase algorithm for planning a trajectory that moves the chained form system from any initial configuration $\boldsymbol{z}_i$ to any final configuration $\boldsymbol{z}_f$. Moreover:

(a) provide a pseudocode description of the algorithm;
(b) discuss which are its free parameters and how to choose their values;
(c) assuming $\boldsymbol{z}_i = (0, 0, 0)$ and $\boldsymbol{z}_f = (1, 1, 1)$, compute the total time needed for reaching $\boldsymbol{z}_f$ and draw the trajectory of the system on the z_1, z_2 plane.

3.1 Setting

The $(2, 3)$ chained form has two inputs and three states:

$$\dot{z}_1 = v_1, \qquad \dot{z}_2 = v_2, \qquad \dot{z}_3 = z_2 \, v_1. \tag{15}$$

We are given the fact that the sinusoidal inputs

$$v_1 = a \sin \omega t, \qquad v_2 = b \cos \omega t$$

applied over $[0, 2\pi/\omega]$ bring z_1, z_2 back to their initial values and produce a displacement $\Delta z_3 = ab\pi/\omega^2$. We exploit it to design a two-phase algorithm moving $\boldsymbol{z}_i = (z_{1,i}, z_{2,i}, z_{3,i})$ to any $\boldsymbol{z}_f = (z_{1,f}, z_{2,f}, z_{3,f})$.

Controllability check. With $\boldsymbol{g}_1 = (1, 0, z_2)^T$ and $\boldsymbol{g}_2 = (0, 1, 0)^T$ we get $[\boldsymbol{g}_1, \boldsymbol{g}_2] = -\partial \boldsymbol{g}_1/\partial z_2 = (0, 0, -1)^T$, so $\operatorname{rank}(\boldsymbol{g}_1, \boldsymbol{g}_2, [\boldsymbol{g}_1, \boldsymbol{g}_2]) = 3$ everywhere: the system is controllable, so the planning problem is always solvable.

Intuition

The first two equations are *pure integrators*: steering (z_1, z_2) anywhere is trivial. The only difficulty is z_3, which is not directly driven: it accumulates $z_2\,\dot{z}_1$. By Green's theorem, over a closed loop in the (z_1, z_2) plane

$$\Delta z_3 = \oint z_2 \, \mathrm{d}z_1 = \pm \text{ (area enclosed by the loop)},$$

with $+$ for a clockwise loop. The sinusoidal inputs trace an ellipse with semi-axes a/ω and b/ω, whose area is $\pi \frac{a}{\omega}\frac{b}{\omega} = \frac{ab\pi}{\omega^2}$, exactly the given displacement. So z_3 is "the area you have swept": *to change z_3, go around a loop in the z_1–z_2 plane.* This is the same idea as the $\Delta\theta$ produced by the leg cycle of the hopping robot in Section 2, and of parallel parking.

3.2 (a) The algorithm

Phase 1 (move z_1, z_2). Drive z_1, z_2 to $z_{1,f}, z_{2,f}$ in a finite time T_1, ignoring z_3. At the end, z_3 has some value $z_3(T_1)$.

Phase 2 (fix z_3). Apply the sinusoidal inputs, shifted in time, with parameters chosen so that the displacement of z_3 equals $z_{3,f} - z_3(T_1)$. Since z_1, z_2 return to $z_{1,f}, z_{2,f}$, the final configuration is $\boldsymbol{z}_f$.

Phase 1 with sign inputs. Set $v_1 = \operatorname{sign}(z_{1,f} - z_{1,i})$, $v_2 = \operatorname{sign}(z_{2,f} - z_{2,i})$ for $t \in [0, T_1]$, with $\operatorname{sign}(0) = 0$ and $T_1 = \max\{|z_{1,f} - z_{1,i}|, |z_{2,f} - z_{2,i}|\}$.

Remark 3.1 (Caution on the open-loop sign inputs). With a *constant* unit-magnitude input over a common time T_1, the component with the smaller displacement would overshoot its target: this version is exact only if $|\Delta z_1| = |\Delta z_2|$ (as in part (c)). Two ways to fix it:

- **feedback sign controller**: $v_1 = \operatorname{sign}(z_{1,f} - z_1)$, $v_2 = \operatorname{sign}(z_{2,f} - z_2)$, so that each input switches off *automatically* when $\operatorname{sign}(0) = 0$ is reached (each component stops at its target, in time $|\Delta z_i|$);
- **time scaling**: $v_i = \Delta z_i / T_1$ over $[0, T_1]$ with any chosen $T_1 > 0$, so that both components arrive simultaneously. This also yields the closed form

$$z_3(T_1) = z_{3,i} + z_{2,i}\Delta z_1 + \tfrac{1}{2}\Delta z_1 \Delta z_2, \tag{16}$$

  obtained by integrating $\dot{z}_3 = z_2 v_1$.

A third alternative is a **proportional feedback** $v_1 = k_1(z_{1,f} - z_1)$, $v_2 = k_2(z_{2,f} - z_2)$, which is smoother but converges only asymptotically; one then switches to Phase 2 once the error is below a threshold, and uses the *measured* value of z_3 to compute the Phase 2 parameters, which makes the scheme robust to the imperfect Phase 1.

Algorithm: two-phase planner for the $(2, 3)$ chained form

Require: $\boldsymbol{z}_i = (z_{1,i}, z_{2,i}, z_{3,i})$, $\boldsymbol{z}_f = (z_{1,f}, z_{2,f}, z_{3,f})$

Phase 1

1: $\Delta_1 \leftarrow z_{1,f} - z_{1,i}$, $\Delta_2 \leftarrow z_{2,f} - z_{2,i}$, $T_1 \leftarrow \max(|\Delta_1|, |\Delta_2|)$ ▷ any $T_1 > 0$ if time-scaled

2: apply $v_1 = \operatorname{sign}(\Delta_1)$, $v_2 = \operatorname{sign}(\Delta_2)$ on $[0, T_1]$ (or the feedback/scaled variant)

3: $z_3(T_1) \leftarrow$ integrate $\dot{z}_3 = z_2 v_1$ (closed form (16) for the scaled variant)

Phase 2

4: $\Delta_3 \leftarrow z_{3,f} - z_3(T_1)$
5: choose $\omega > 0$ and $a \neq 0$; set $b \leftarrow \dfrac{\omega^2 \Delta_3}{\pi a}$ ▷ so that $ab\pi/\omega^2 = \Delta_3$
6: apply $v_1 = a \sin \omega(t - T_1)$, $v_2 = b \cos \omega(t - T_1)$ for $t \in [T_1,\, T_1 + 2\pi/\omega]$
7: **return** total time $T = T_1 + 2\pi/\omega$

3.3 (b) Free parameters and how to choose them

Phase 1 contains no parameters (apart from the optional T_1 in the scaled variant). Phase 2 has three parameters (a, b, ω) subject to *one* constraint $ab\pi/\omega^2 = z_{3,f} - z_3(T_1)$, so two remain free.

- a, b are the **input amplitudes**: they should be chosen compatibly with the bounds on the available inputs. They also shape the trajectory: (z_1, z_2) traces an ellipse with semi-axes a/ω (along z_1) and b/ω (along z_2), so $|a| > |b|$ gives an ellipse elongated along z_1 and $|a| < |b|$ along z_2.
- ω sets the **duration** $2\pi/\omega$ of Phase 2, hence the total time.
- The sign of ab is fixed by the sign of Δ_3: $\Delta_3 > 0$ requires $ab > 0$ (clockwise loop), $\Delta_3 < 0$ requires $ab < 0$ (counter-clockwise).

Intuition

There is a trade-off between speed and effort. For the product ab fixed by the area to be swept, the peak input $\max(|a|, |b|)$ is smallest when $|a| = |b|$ (a circle): any elongation raises one amplitude. For $|a| = |b|$ the area constraint gives $a^2 = \Delta_3\, \omega^2/\pi$, i.e. $a \propto \omega$: going faster (larger ω) needs proportionally larger inputs. With a bound $v_{\max}$ on the inputs, the fastest feasible loop has $|a| = v_{\max}$.

3.4 (c) The case $\boldsymbol{z}_i = (0, 0, 0)$, $\boldsymbol{z}_f = (1, 1, 1)$

Phase 1: $\Delta_1 = \Delta_2 = 1$, so $T_1 = \max(1, 1) = 1$. The inputs $v_1 = v_2 = 1$ drive (z_1, z_2) along the straight line from $(0, 0)$ to $(1, 1)$ (here the open-loop sign version is exact). At the end,

$$z_3(T_1) = \int_0^1 z_2 v_1 \, \mathrm{d}t = \int_0^1 t \, \mathrm{d}t = \tfrac{1}{2},$$

in agreement with (16). Phase 2 must therefore produce $\Delta_3 = 1 - \frac{1}{2} = \frac{1}{2}$, i.e. $ab\pi/\omega^2 = \frac{1}{2}$.

With ω chosen arbitrarily, the total time is

$$T = 1 + \frac{2\pi}{\omega}.$$

In the (z_1, z_2) plane the trajectory is the segment from $(0, 0)$ to $(1, 1)$, followed by the ellipse of Phase 2 closing at $(1, 1)$. For instance, with $a = b$ one has $a = \omega/\sqrt{2\pi}$ and the loop is a circle of radius $a/\omega = 1/\sqrt{2\pi} \approx 0.40$ centered at $(1 + 0.40,\, 1)$; with $\omega = 2\pi$ the amplitudes are $a = b = \sqrt{2\pi} \approx 2.51$ and the total time is $T = 2$.

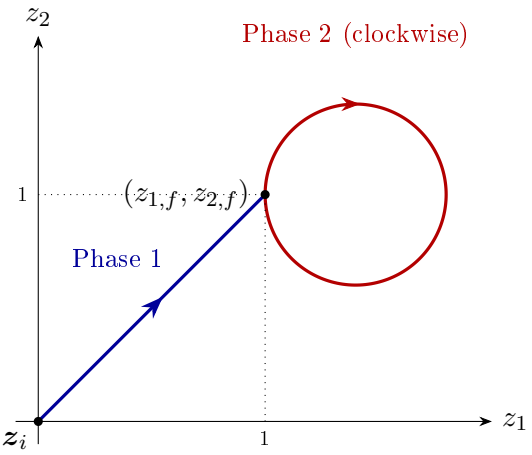


Figure 5: Trajectory in the (z_1, z_2) plane for $\boldsymbol{z}_i = (0,0,0)$, $\boldsymbol{z}_f = (1,1,1)$ with $a = b$: a line (Phase 1) followed by a circle (Phase 2). The area enclosed by the circle, $\pi/(2\pi) = 1/2$, is exactly the missing Δz_3.

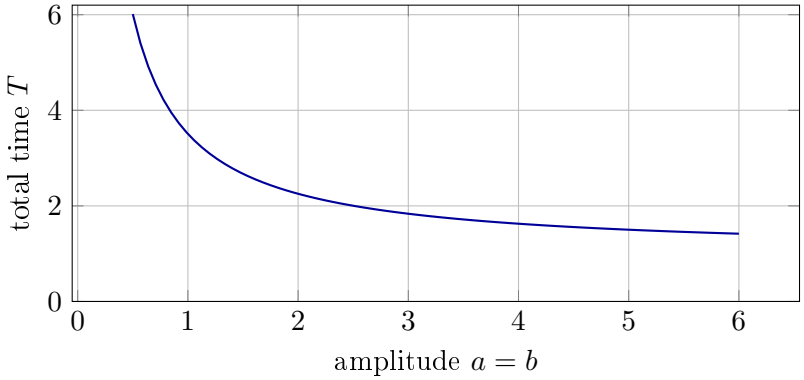


Figure 6: Trade-off for $\boldsymbol{z}_i = (0,0,0) \to \boldsymbol{z}_f = (1,1,1)$ with $a = b$: $T = 1 + \sqrt{2\pi}/a$. Larger input amplitude shortens Phase 2, which can never be shorter than zero, so $T \to T_1 = 1$.

4 Front-Wheel-Drive Bicycle: Modelling, Flatness and Control

(Midterm Class Test 2024/25, Problem 1)

Problem statement

Consider a bicycle robot of length ℓ with front-wheel drive. Define the configuration vector as $\boldsymbol{q} = (x_f, y_f, \theta, \phi)$, where (x_f, y_f) are the Cartesian coordinates of the (contact point of the) front wheel, θ is the vehicle orientation, and ϕ is the steering angle of the front wheel.

(a) Starting from the kinematic constraints acting on the system, derive a kinematic model of the vehicle with the front wheel driving and steering velocities as control inputs, and prove its controllability.

(b) Are x_f, y_f flat outputs for this model? Motivate your answer and give an interpretation in terms of state/input reconstructability.

(c) Design a feedback controller for driving x_f, y_f along a desired trajectory $x_f^*(t), y_f^*(t)$ and provide the corresponding block scheme. *Hint: find an invertible mapping between the time derivative (of a suitable order) of the outputs and the dynamically extended control inputs.*

4.1 Setting

Consider a bicycle robot of length ℓ with front-wheel drive and configuration vector $\boldsymbol{q} = (x_f, y_f, \theta, \phi)^T$, where (x_f, y_f) are the Cartesian coordinates of the contact point of the front wheel, θ is the vehicle orientation and ϕ is the steering angle of the front wheel.

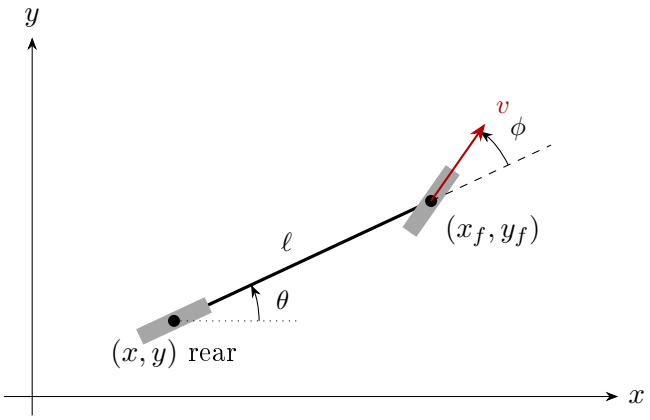


Figure 7: Front-wheel-drive bicycle. The front wheel has orientation $\theta + \phi$ and carries both drive (v) and steering ($\omega = \dot{\phi}$) inputs.

4.2 (a) Kinematic model and controllability

Constraints. There is one pure-rolling condition per wheel (no lateral slip):

$$\dot{x}\sin\theta - \dot{y}\cos\theta = 0 \qquad \text{(rear wheel)} \tag{17}$$

$$\dot{x}_f \sin(\theta+\phi) - \dot{y}_f \cos(\theta+\phi) = 0 \qquad \text{(front wheel)}, \tag{18}$$

where (x, y) are the coordinates of the rear wheel contact point, related to the front one by

$$x = x_f - \ell\cos\theta, \qquad y = y_f - \ell\sin\theta.$$

Differentiating, $\dot{x} = \dot{x}_f + \ell\dot{\theta}\sin\theta$, $\dot{y} = \dot{y}_f - \ell\dot{\theta}\cos\theta$, and substituting into the rear constraint ($\sin^2\theta + \cos^2\theta = 1$):

$$\dot{x}_f \sin\theta - \dot{y}_f \cos\theta + \ell\dot{\theta} = 0.$$

In Pfaffian form, with $\boldsymbol{q} = (x_f, y_f, \theta, \phi)$,

$$A^T(\boldsymbol{q})\dot{\boldsymbol{q}} = \begin{pmatrix} \sin\theta & -\cos\theta & \ell & 0 \\ \sin(\theta+\phi) & -\cos(\theta+\phi) & 0 & 0 \end{pmatrix} \begin{pmatrix} \dot{x}_f \\ \dot{y}_f \\ \dot{\theta} \\ \dot{\phi} \end{pmatrix} = 0. \tag{19}$$

Here $n = 4$ and $k = 2$, so the null space is 2-dimensional.

Null-space basis and model. A basis is

$$\boldsymbol{g}_1(\boldsymbol{q}) = \begin{pmatrix} \cos(\theta+\phi) \\ \sin(\theta+\phi) \\ \frac{\sin\phi}{\ell} \\ 0 \end{pmatrix}, \qquad \boldsymbol{g}_2(\boldsymbol{q}) = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}.$$

(Check: the second row of A^T annihilates $\boldsymbol{g}_1$ because $\sin(\theta+\phi)\cos(\theta+\phi) - \cos(\theta+\phi)\sin(\theta+\phi) = 0$; the first row gives $\sin\theta\cos(\theta+\phi) - \cos\theta\sin(\theta+\phi) + \sin\phi = -\sin\phi + \sin\phi = 0$.) The kinematic

model is

$$\boxed{\begin{aligned} \dot{x}_f &= \cos(\theta+\phi)\, v \\ \dot{y}_f &= \sin(\theta+\phi)\, v \\ \dot{\theta} &= \frac{\sin\phi}{\ell}\, v \\ \dot{\phi} &= \omega \end{aligned}} \tag{20}$$

where v and ω are the driving and steering velocities, both generated at the front wheel.

Intuition

The front wheel can only move along its own heading $\theta+\phi$ (this is v). The rear wheel must follow, it cannot slip sideways, so the body turns at a rate $\frac{v\sin\phi}{\ell}$: it is the component of v perpendicular to the body, divided by the lever arm ℓ. If $\phi = 0$ the vehicle goes straight; the larger the steering angle, the tighter the turn; at $\phi = \pm\pi/2$ the front wheel moves perpendicular to the body and the body pivots about the rear wheel. Steering is a separate, directly actuated velocity $\omega = \dot{\phi}$.

Controllability. We compute the Lie brackets:

$$\boldsymbol{g}_3 = [\boldsymbol{g}_1, \boldsymbol{g}_2] = \begin{pmatrix} \sin(\theta+\phi) \\ -\cos(\theta+\phi) \\ -\frac{\cos\phi}{\ell} \\ 0 \end{pmatrix}, \qquad \boldsymbol{g}_4 = [\boldsymbol{g}_1, \boldsymbol{g}_3] = \begin{pmatrix} -\frac{\sin\theta}{\ell} \\ \frac{\cos\theta}{\ell} \\ 0 \\ 0 \end{pmatrix}.$$

Then, reordering the columns,

$$\operatorname{rank}\begin{pmatrix} \boldsymbol{g}_1 & \boldsymbol{g}_3 & \boldsymbol{g}_4 & \boldsymbol{g}_2 \end{pmatrix} = \operatorname{rank}\begin{pmatrix} \cos(\theta+\phi) & \sin(\theta+\phi) & -\frac{\sin\theta}{\ell} & 0 \\ \sin(\theta+\phi) & -\cos(\theta+\phi) & \frac{\cos\theta}{\ell} & 0 \\ \frac{\sin\phi}{\ell} & -\frac{\cos\phi}{\ell} & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} = 4,$$

because the determinant of the upper-left 3×3 block is $1/\ell^2 \neq 0$ (expand along the third column and use $\sin(\theta+\phi)\cos\theta - \cos(\theta+\phi)\sin\theta = \sin\phi$ and $\cos(\theta+\phi)\cos\theta + \sin(\theta+\phi)\sin\theta = \cos\phi$). The rank is 4 for every $\boldsymbol{q}$, so the kinematic model is *controllable*.

Intuition

The bracket $\boldsymbol{g}_3$ is the "wiggle the steering" direction (drive a bit, steer, drive back, steer back), and $\boldsymbol{g}_4$ is the "lateral" direction perpendicular to the rear wheel. This is precisely the sideways motion that the constraint forbids directly but parallel parking obtains: the determinant $1/\ell^2$ never vanishes, so there is no configuration (not even $\phi = \pm\pi/2$) where this fails.

4.3 (b) Are x_f, y_f flat outputs?

What can be reconstructed. From the first two equations of (20),

$$\theta + \phi = \arctan\frac{\dot{y}_f}{\dot{x}_f}, \qquad v = \pm\sqrt{\dot{x}_f^2 + \dot{y}_f^2}$$

(the sign of v distinguishes forward from backward motion), and differentiating the first,

$$\dot{\theta} + \dot{\phi} = \frac{\ddot{y}_f\dot{x}_f - \dot{y}_f\ddot{x}_f}{\dot{x}_f^2 + \dot{y}_f^2} =: \kappa, \qquad \text{so} \quad \dot{\theta} = \kappa - \dot{\phi}. \tag{21}$$

The obstruction. From the third equation of (20), $\sin\phi = \ell\dot{\theta}/v$, hence

$$\phi = \arcsin\left[\frac{\ell}{v}\left(\frac{\ddot{y}_f\dot{x}_f - \dot{y}_f\ddot{x}_f}{\dot{x}_f^2 + \dot{y}_f^2} - \dot{\phi}\right)\right], \qquad \text{equivalently} \qquad \dot{\phi} = \kappa - \frac{v}{\ell}\sin\phi. \tag{22}$$

The right-hand side contains $\dot{\phi}$ itself: ϕ is the solution of a *differential equation* driven by the outputs, not an algebraic function of them. Therefore ϕ (and consequently $\theta = \arctan(\dot{y}_f/\dot{x}_f) - \phi$) *cannot be reconstructed algebraically* from x_f, y_f and their derivatives:

$$\boxed{x_f, y_f \text{ are } \textit{not} \text{ flat outputs.}}$$

Interpretation (state/input reconstructability). Along a given front-wheel trajectory, v and the heading $\theta + \phi$ of the front wheel are known, but the *split* of this heading between body orientation θ and steering ϕ depends on the initial condition of (22). For the same front-wheel path there are infinitely many possible rear-wheel paths, one per initial condition.

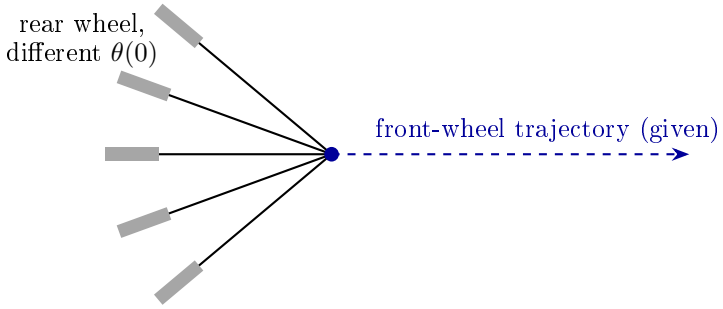


Figure 8: The same front-wheel path is compatible with infinitely many rear-wheel configurations, depending on the initial body orientation: this is why (x_f, y_f) do not determine the state.

Intuition

It is a matter of what the front point alone can tell you. Suppose you only watch the front wheel moving along a straight line: is the car perfectly aligned behind it, or is the body at an angle, with the steering compensating? Both are consistent with the same front-wheel path. Which one is true is "memory" stored in ϕ, and memory means integration, not algebra. Equation (22) adds a further insight: for forward motion $v > 0$ and $|\phi| < \pi/2$, the linearization $-\frac{v}{\ell}\cos\phi$ is negative, so the ϕ-dynamics are *stable* (the body aligns behind the front wheel, like a shopping cart caster); for backward motion the same ODE is *unstable* (jackknifing).

Remark 4.1. A different choice of output *does* work: for the bicycle with front-wheel drive, the Cartesian position of the *rear* wheel is a flat output, because θ is recovered from the direction of its velocity (the rear wheel moves along the body axis), and ϕ, v then follow algebraically. The non-flatness above is specific to the front-wheel point.

4.4 (c) Tracking controller for (x_f, y_f): dynamic feedback linearization

Differentiate the outputs. The first derivatives are $\dot{x}_f = \cos(\theta+\phi)v$, $\dot{y}_f = \sin(\theta+\phi)v$: only v appears, so no decoupling is possible at this order (one input, two outputs). Differentiating again,

$$\begin{aligned}\ddot{x}_f &= -\sin(\theta+\phi)(\dot{\theta}+\dot{\phi})\,v + \cos(\theta+\phi)\,\dot{v},\\ \ddot{y}_f &= \;\;\cos(\theta+\phi)(\dot{\theta}+\dot{\phi})\,v + \sin(\theta+\phi)\,\dot{v}.\end{aligned}$$

Dynamic extension. Set $\dot v = a$ (the acceleration becomes the input, and v becomes a state), and replace $\dot\theta = \frac{\sin\phi}{\ell} v$, $\dot\phi = \omega$:

$$\begin{pmatrix} \ddot x_f \\ \ddot y_f \end{pmatrix} = \underbrace{\begin{pmatrix} -\sin(\theta+\phi)\frac{\sin\phi}{\ell} v^2 \\ \cos(\theta+\phi)\frac{\sin\phi}{\ell} v^2 \end{pmatrix}}_{\boldsymbol{b}(\boldsymbol{q},v)} + \underbrace{\begin{pmatrix} \cos(\theta+\phi) & -\sin(\theta+\phi)\, v \\ \sin(\theta+\phi) & \cos(\theta+\phi)\, v \end{pmatrix}}_{\boldsymbol{T}(\boldsymbol{q},v)} \begin{pmatrix} a \\ \omega \end{pmatrix}. \tag{23}$$

The decoupling matrix has $\det \boldsymbol{T} = v \neq 0$: it is invertible whenever $v \neq 0$. In this case

$$\boldsymbol{T}^{-1}(\boldsymbol{q}, v) = \frac{1}{v} \begin{pmatrix} v\cos(\theta+\phi) & v\sin(\theta+\phi) \\ -\sin(\theta+\phi) & \cos(\theta+\phi) \end{pmatrix}.$$

Linearizing law and tracking. Let

$$\begin{pmatrix} a \\ \omega \end{pmatrix} = \boldsymbol{T}^{-1}(\boldsymbol{q}, v) \left(\begin{pmatrix} u_1 \\ u_2 \end{pmatrix} - \boldsymbol{b}(\boldsymbol{q}, v) \right). \tag{24}$$

The system from (u_1, u_2) to the outputs is then linear and decoupled,

$$\ddot x_f = u_1, \qquad \ddot y_f = u_2.$$

For a desired smooth trajectory $x_f^*(t), y_f^*(t)$ use the PD+feedforward law

$$u_1 = \ddot x_f^* + k_{p1}(x_f^* - x_f) + k_{d1}(\dot x_f^* - \dot x_f), \tag{25}$$
$$u_2 = \ddot y_f^* + k_{p2}(y_f^* - y_f) + k_{d2}(\dot y_f^* - \dot y_f). \tag{26}$$

With $e_x = x_f^* - x_f$ the error obeys $\ddot e_x + k_{d1}\dot e_x + k_{p1} e_x = 0$, whose characteristic polynomial $s^2 + k_{d1}s + k_{p1}$ is Hurwitz iff $k_{p1}, k_{d1} > 0$; the same holds for y. Hence *global exponential tracking* is obtained for any positive gains, as long as $v \neq 0$.

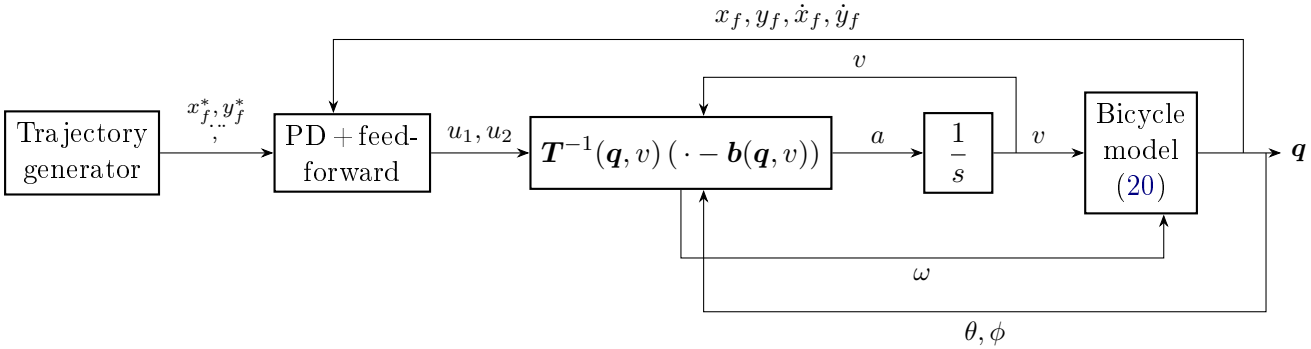


Figure 9: Block scheme of the dynamic feedback linearizing controller for the front wheel. The integrator turns the acceleration input a into the driving velocity v, so the controller is intrinsically dynamic (its state is v).

Practical remarks.

- To compute u_1, u_2 one needs x_f, y_f and their first derivatives. These can be obtained from the original measured states (x, y, θ or x_f, y_f, θ, ϕ), and in fact *without numerical differentiation*, since $\dot x_f = \cos(\theta+\phi)v$ and $\dot y_f = \sin(\theta+\phi)v$ are known as soon as θ, ϕ, v are.
- To compute (a, ω) through (24), the angles θ, ϕ and the extended state v are needed.
- The driving velocity v is obtained *by integrating* a: the controller has an internal state, and must be initialized with $v(0) \neq 0$.

Intuition

Why do we need to add an integrator? At the level of $\ddot{x}_f, \ddot{y}_f$ we have two outputs but, as long as v is regarded as an input, a single actuator in the first derivatives. Delaying the appearance of v by one step (treating v as a state and a as the input) gives two inputs (a, ω) that both show up in the second derivatives, and the matrix that multiplies them is invertible provided the vehicle is moving. The condition $v \neq 0$ has a clear physical meaning: when the front wheel is at rest it cannot instantaneously accelerate *sideways*: the steering input ω affects the front-wheel acceleration only through v ($\omega\, v$ terms). Finally, the linearized dynamics have order $2+2=4$ whereas the extended system has order 5: the remaining state is the unobserved internal dynamics which is, again, the steering equation (22), stable for forward motion. This is consistent with part (b).

5 Summary

5.1 Comparison of the three problems

	Hopping robot	$(2,3)$ **chained form**	**Front-wheel bicycle**
State / inputs	$\boldsymbol{q} = (\phi, \ell, \theta)$; $u_1 = \dot{\phi}$, $u_2 = \dot{\ell}$	$\boldsymbol{z} = (z_1, z_2, z_3)$; v_1, v_2	$\boldsymbol{q} = (x_f, y_f, \theta, \phi)$; v, ω
n, k	3, 1 (angular momentum)	3, 0 (already a model)	4, 2 (rolling)
Controllable?	Yes: $[\boldsymbol{g}_1, \boldsymbol{g}_2] = (0, 0, \frac{2m(d+\ell)}{J})^T$	Yes: $[\boldsymbol{g}_1, \boldsymbol{g}_2] = (0, 0, -1)^T$	Yes: $\boldsymbol{g}_1, \boldsymbol{g}_2, [\boldsymbol{g}_1, \boldsymbol{g}_2], [\boldsymbol{g}_1, [\boldsymbol{g}_1, \boldsymbol{g}_2]]$ span, $\det = 1/\ell^2$
Flatness	Flat for $d = 0$; outputs (ϕ, θ)	(Chain form is flat: z_1, z_3)	(x_f, y_f) *not* flat (ϕ via ODE)
Main tool	geometric model, polynomial interpolation of flat outputs	Two phases: integrate z_1, z_2; loop to fix z_3	Dynamic extension $\dot{v} = a$, $\boldsymbol{T}^{-1}$
Result	Path planning; regulator $\theta \to 0$, $\ell \to h$	$T = T_1 + 2\pi/\omega$; $ab\pi/\omega^2 = \Delta z_3$	Exponential tracking of x_f^*, y_f^* if $v \neq 0$
Where it fails	$d+\ell = 0$; $\dot{\phi} = 0$; θ' must not change sign	$\omega \to 0$ (slow) or large a, b	$v = 0$

Table 1: Side-by-side view of the three exercises.

5.2 Key points

1. **A velocity constraint is not a loss of reachability.** In all three systems the constraint is nonholonomic; brackets of the input vector fields (a leg cycle, a steering wiggle, an area swept in the (z_1, z_2) plane) supply the "missing" directions, and the rank condition certifies it.

2. **Choose the basis for interpretability.** The null space of the constraint admits many bases; take the one whose inputs are directly actuated ($\dot{\phi}, \dot{\ell}$ for the hopper; v, ω for the bicycle).

3. **Flatness is a statement about algebra versus integration.** If a hidden variable is obtained by an algebraic formula (as ℓ from θ'/ϕ'), the output is flat and planning reduces

to polynomial interpolation. If it requires solving an ODE (as ϕ from the front-wheel path), it is not flat: the same output curve hides infinitely many states.

4. **Closed loops in the base variables generate net motion in the fiber.** The $\Delta\theta$ of the leg cycle and Δz_3 equal to the enclosed area are the same phenomenon, and give simple two-phase planners.

5. **When the outputs are not flat, dynamic extension can still give exact tracking**, at the price of a singularity ($v = 0$) and of leaving an internal dynamics (the steering angle) that must be stable.